

Homework 1

Due: September 23rd @ 11:55 pm on Gradescope

Name: [Shikha Nukala](#) Uniqname: [desigirl](#)

1. Submit a pdf of your typed or handwritten homework on Gradescope.
2. Your answers should be neat, clearly marked, and concise. Typed work is recommended, but not required unless otherwise stated. Show all your work where requested, and state any special or non-obvious assumptions you make.
3. You may discuss your solution methods with other students, but the solutions you submit must be your own.
4. **Late Homework Policy:** Submissions turned in by 1:00 am the next day will be accepted but with a 5% penalty. Assignments turned in between 1:00 am and 11:55 pm will get a 30% penalty, and any submissions made after this time will not be accepted.
5. When submitting your answers to Gradescope you need to indicate what page(s) each problem is on to receive credit. The grader may choose not to grade the homework if answer locations are not indicated.
6. After each question (or in some cases question part), we've indicated which lecture number we expect to cover the relevant material. So "(L7)" indicates that we expect to cover the material in lecture 7.
7. **The last question is a group question.**
 - You may do it in a homework group of up to two students including yourself (yes, you can do them by yourself if you wish).
 - If you work in a group of two for these questions, list the name of the student you worked with in your assignment. Further, we suggest that you not split these problems up but rather work on the problem as a group.
 - Turn these group questions in as part of your individual submission.
 - **For these questions (and these questions ONLY) you are allowed to copy/paste solutions from the other student in your homework group.**
 - It is an honor code violation if a student is listed as contributing who did not actually participate in working on that problem.

Problem 1 (20 points): Unsigned binary numbers and shifting (L2)

1. Convert the following (unsigned) binary numbers to decimal. [4]

a. 0b10110

$$2^1 + 2^2 + 2^4 = 2 + 4 + 16 = 22$$

b. 0b001001

$$2^0 + 2^3 = 1 + 8 = 9$$

2. Convert the following decimal numbers to 8-bit binary. [6]

a. $6 = 2^2 + 2^1 = 4 + 2$

0b00000110

b. $20 = 2^4 + 2^2 = 16 + 4$

0b00010100

c. $110 = 2^6 + 2^5 + 2^3 + 2^2 + 2^1 = 64 + 32 + 8 + 4 + 2$

0b01101110

3. Answer the following (EDIT: assume all binary numbers are unsigned):

a. What is the decimal representation of the binary number 0b110? [2] $2^1 + 2^2 = 6$

b. What is the decimal representation of the binary number 0b1100? [2] $2^3 + 2^2 = 12$

c. In general, what happens to the value of a binary number when you add a zero to the end of it (least significant)? [3]

The decimal value of the number is doubled because the number is shifted by a power of 2 so it is multiplied by an additional 2.

d. When you add 4 zeros to the end? [3]

Adding 4 zeros would double the number four times or multiply the decimal value by 16 since the binary number is shifted 4 bits which is 4 powers of 2 (2^4).

Problem 2 (20 points): Reading LC2K Code (L3)

This problem will get you some exposure into the correspondence between C code and assembly, specifically the LC2K instruction set architecture.

Copy-paste the following code into the "Instruction Memory" window of the LC2K simulator [linked on the website](#):

```
lw      0      1      index
lw      1      2      array
add     2      2      3
sw      1      3      array
halt
index   .fill   2
array   .fill   7
        .fill   -2
        .fill   16
        .fill   3
```

Click "verify" and you should see the following window, showing the initial values of the register file (all zero), and the contents of data memory (initialized to the values specified by .fill directives in the program). (*Note, if you get an error at this step, try copy-pasting the above code into a plain text editor, and then copying it into the instruction window. Make sure to include the leading white space on the first line.)

The screenshot displays the LC2K simulator interface with three main windows:

- Instruction Memory:** Contains the assembly code as provided in the problem statement, including the .fill directives.
- Register File:** A vertical list of registers R0 through R7, each with a value of 0.
- Data Memory:** A vertical list of memory locations 5 through 9, with values 2, 7, -2, 16, and 3 respectively.

The simulator interface includes a "Load Instructions" dropdown menu, a "Reset" button, and "Step" and ">>" buttons.

You can now click "Step >" to see the effect of each instruction executing. You should see either one of the register values changing or one of the lines in data memory changing after each instruction

1. Explain, line-by-line, what this program is doing with each instruction executed (you don't need to explain the .fill directives, those aren't executed instructions. They just initialize the contents of data memory when we load the program). It should be clear from your explanation what each operand listed is doing. [10]

```
lw      0      1      index
```

This line loads a value into register 1. Since the first register is 0, this line simply loads the value at index into register 1. Index holds the value 2 so 2 is loaded into register 1.

```
lw      1      2      array
```

This line loads a value into register 2. The value is obtained by finding the value at memory location $\text{reg}[1] + \text{location of array}$. $\text{Reg}[1]$ holds the value of 2 and the memory location of array is 6. $2+6 = 8$ and the 8th instruction memory location (starting at 0) holds the value of 16, so 16 is loaded into register 2.

```
add     2      2      3
```

This line adds the first two registers and loads the result into register 3. The value at $\text{reg}[2]$ is 16 so this operation does $\text{reg}[2] + \text{reg}[2] = 16 + 16 = 32$. Therefore 32 is loaded into register 3.

```
sw      1      3      array
```

This line stores what is in register 3 to the memory location of the value in $\text{reg}[1]$ (2) + starting memory location of array (6). This is $2 + 6$ which is 8. The value at $\text{reg}[3]$ is 32, so in memory location 8, 32 is stored.

```
halt
```

This line halts the program so no more instructions are executed.

2. Write a C program that has the same functionality as the assembly code above. Assume all values are signed integers, and that values initialized in the assembly code using .fill directives are all initialized as global variables with the same labels used in the assembly program. [10]

```
int index = 2; //variable index is filled with 2
```

```
int array[] = {7, -2, 16, 3}; //array holds 4 values
```

```
int main() {
```

```
    int registers[8] = {0}; //represent registers 0-7 initially set to 0
```

```
    int offset; //to set offset for lw, sw, beq
```

```
    registers[1] = index; // lw 0 1 index
```

```
    offset = registers[1]; //offset is 2 now
```

```
    registers[2] = array[offset]; //lw 1 2 array loads 16 to register 2
```

```
    registers[3] = registers[2] + registers[2]; //add 2 2 3 loads 32 to register 3
```

```
    offset = registers[1]; //set offset to value at register 1
```

```
    array[offset] = registers[3]; //change array at offset to register 3
```

```
    return 0; //program halted
```

```
}
```

Problem 3 (20 Points): Moving data between memory and registers (L5)

For the following LEGv8 assembly code, indicate the final state of the registers and memory locations listed once this code has finished. Assume any memory address outside of that displayed in the initial memory state contains 0.

```
MOVZ      X0, #0x1001 //moves 0x1001 into X0
LDURH     X2, [X4, #0] //load 2 bytes of mem(X4+0) into X2
STURB     X3, [X0, #3] //store byte FF from X3 into X0+3 0x1004
LDURSW    X4, [X0, #-1] //32 bit word from 0-3 in memory
STURH     X4, [X0, #1] //mem at 2 is 04, 3 is 12
```

Initial Register State		Initial Memory State	
Register	Value	Address	Value
X0	0x5	0x00001000	0x4
X1	0x1000	0x00001001	0x12
X2	0x6E	0x00001002	0x33
X3	0x41FF	0x00001003	0x2E
X4	0x1002	0x00001004	0xA1
X5	0x0	0x00001005	0x77

- 1) Fill in the following table. Express your final answers in hexadecimal. Assume memory is organized in a **little endian** manner. [10]

Final Register State		Final Memory State	
Register	Value	Address	Value
X0	0x1001	0x00001000	0x4
X1	0x1000	0x00001001	0x12
X2	0x2E33	0x00001002	0x04
X3	0x41FF	0x00001003	0x12
X4	0x2E331204	0x00001004	0xFF
X5	0x0	0x00001005	0x77

2) Fill in the following table. Express your final answers in hexadecimal. Assume memory is organized in a **big endian** manner. **[10]**

Final Register State		Final Memory State	
Register	Value	Address	Value
X0	0x1001	0x00001000	0x4
X1	0x1000	0x00001001	0x12
X2	0x332E	0x00001002	0x33
X3	0x41FF	0x00001003	0x2E
X4	0x0412332E	0x00001004	0xFF
X5	0x0	0x00001005	0x77

Problem 4: LEGv8 (20 points)--Place your answers in the appropriate box. (L6)

Variable to register mappings

int64_t t - X2 int64_t v - X4 int64_t x - X7
int64_t u - X3 int64_t w - X5 int64_t y - X8

- 1) Convert the following C code to no more than 4 lines of equivalent LEGv8 assembly, assume `int64_t arr[]`'s base address is held in `x6`. Additionally use register `x1` as a temporary register. Recall that unlike LC2K, the LEGv8 ISA is byte addressed. [10]

```
w = arr[x];  
arr[x+7] = y;
```

```
LSL    X1, X7, #3 //shift 8 for array  
LDUR   X5, [X6, X1] //load arr[x] into w  
ADD    X1, X1, #56 //offset temp, 7*8 shift  
STUR   X8, [X6, X1] //store y into arr[x+7]
```

- 2) Convert the following C code to no more than 5 lines of LEGv8 assembly. Please use **CBZ** or **CBNZ** for your conditional instruction. [10]

```
while(x != 0){  
    x--;  
    y+=x;  
}
```

```
Loop    CBNZ    X7    Goto  
        B        End  
Goto    SUB     X7, X7, #1  
        ADD     X8, X7, X8  
        B        Loop  
End
```

Problem 5 (20 points, group): Transistor Performance (L1)

You must **type** the answer to these questions. Read about Moore's Law and Dennard Scaling on Wikipedia (or some other source if you wish) then answer the following questions. Each answer should be under 100 words.

1. Define Moore's Law in your own words.

Moore's Law states that the number of transistors (the semiconductor that controls electrical signals in any device) on an integrated circuit (a chip with many transistors that is used to process and store information in electronic devices) will double every two years at about the same price as it is today allowing chips in devices to process and store information at a higher space and rate than current chips.

2. Define Dennard Scaling in your own words.

Dennard Scaling is reducing the supply of voltage (that is the amount of power necessary to operate all transistors on a chip) as the size of transistors decrease on a chip (therefore more transistors can be on the chip). This means that with the trend of increasing the number of smaller transistors on a chip that previously had a smaller number of larger transistors, the power density (the power per unit of space on the chip) that the chip takes remains constant.

3. Imagine you have a microprocessor that follows Dennard Scaling perfectly. If you make the chip have twice as many transistors of the same size (thus doubling its area), *roughly* what would you expect would happen to the performance and power consumption of the microprocessor? (We are looking for things like "go up", "go down" "stay the same" etc.) Briefly explain your reasoning.

Since this microprocessor follows Dennard Scaling, the performance of the chip would go up because having twice as many transistors on the same chip would increase the capacity to process and store information on the chip. The power consumption of the chip would go up in total because the area has doubled, so we would expect the power consumption to roughly double. This is because the power density stays the same in Dennard Scaling, so power per unit of space on the chip is constant.

4. Say that the power consumed by a single transistor is constant and you halve the number of transistors on the chip while keeping the area the same (i.e., each transistor becomes bigger). *Roughly* what would you expect would happen to the performance and power consumption of the microprocessor? (We are looking for things like "go up", "go down" "stay the same" etc.) Briefly explain your reasoning.

If the number of transistors is halved then we would expect the performance to go down. Since transistors are proportional to the chip's ability to process and store information, there would be less capability with a lower amount of transistors. For power consumption, if we are considering Dennard Scaling was true for the previous chip, then we can say the power consumption would stay the same because power density is constant in Dennard Scaling and the area of the chip is the same, so for the bigger and smaller transistors the total power consumption would be roughly the same.